Redis Cache Integration

Technical Note | maiproto Node.js + PostgreSQL + Redis Stack

Problem Statement

The User Management API was experiencing performance issues:

- Every `GET /api/users` request queries the entire user list from PostgreSQL
- Database is hit even when data hasn't changed recently
- No caching layer between API and database
- No way to toggle caching on/off without code changes
- Production deployments required either full caching or no caching — no flexibility

Solution Overview

Implement **optional Redis caching** with:

- Cache-aside pattern for read operations (`getUsers` , `getUser`)
- Surgical cache updates on write operations (create, update, delete)
- Environment flag to enable/disable Redis without code changes
- Complete decoupling of cache logic from business logic
- Docker profile-gated Redis service (no forced startup)

Design Architecture

Data Flow

```
Request → userService.js → cache.js → (Redis or null) → Prisma
                                → PostgreSQL
```

Core Components

File	Responsibility
`src/cache/cache.js`	Cache abstraction layer — all Redis operations isolated here
`src/services/userService.js`	Pure business logic, zero knowledge of caching mechanism
`src/server.js`	Conditional Redis connection based on `REDIS_ENABLED` flag
`docker-compose.yml`	Redis service gated behind `--profile redis`

Cache Strategy

Operation	Cache Key	Strategy
`getUsers()`	`users:all`	Cache-aside: miss → fetch all → store → return
`getUser(id)`	`user:5`	Cache-aside: miss → fetch by id → store → return
`createUser()`	`users:all`	Append new user to cached list
`updateUser(id)`	`users:all` + `user:id`	Replace user in list + update individual user cache
`deleteUser(id)`	`users:all` + `user:id`	Filter user from list + delete individual user cache

Cache Configuration

TTL (Time To Live): Default 300 seconds (5 minutes), configurable via `REDIS_TTL` environment variable.

Re-expansion: TTL is re-set on every write operation to maintain fresh expiration.

Environment Variables

```
REDIS_ENABLED=true|false    # Toggle cache on/off
REDIS_HOST=app-redis        # Docker service name for internal ne
REDIS_PORT=6379             # Redis port
REDIS_TTL=300               # Cache expiration in seconds
```

Note: All environment variables must be set in the `.env` file and all 4 `env_templates/.env.{dev,test,demo,prod}` files.

Docker Deployment

With Redis (Caching Enabled)

```
# Setup
copy env_templates\.env.dev .env
# Ensure REDIS_ENABLED=true in .env

# Build and start with Redis
docker compose --profile dev --profile redis build --no-cache
docker compose --profile dev --profile redis up

# Watch logs for cache hits
docker logs nodeapi-dev -f
# Look for: "[Redis] Cache HIT — users:all"

# Stop
docker compose --profile dev --profile redis down
```

Without Redis (Direct Database)

```
# Setup
copy env_templates\.env.dev .env
# Set REDIS_ENABLED=false in .env

# Build and start WITHOUT Redis (no --profile redis)
docker compose --profile dev build --no-cache
docker compose --profile dev up
```

```
# Stop
docker compose --profile dev down
```

> **Behavior:** When `--profile redis` is omitted, Redis service is never started. The `cache.js` wrapper returns `null` on all get operations, forcing every request through Prisma to PostgreSQL.

Switching Modes at Runtime

No rebuild required — just:

1. Stop: `docker compose --profile dev [--profile redis] down`
2. Edit `.env` : toggle `REDIS_ENABLED=true/false`
3. Adjust Docker command based on new flag value
4. Start: `docker compose --profile dev [--profile redis] up`

Testing and Verification

Cache Hit Verification

```
# Terminal 1: Watch logs in real-time
docker logs nodeapi-dev -f

# Terminal 2: Make API requests
# First request (database hit, no cache log)
curl http://localhost:5501/api/users

# Subsequent requests (cache hits, logs appear)
curl http://localhost:5501/api/users
# Expected log: "[Redis] Cache HIT — users:all"

# Create a user (invalidates cache)
curl -X POST http://localhost:5501/api/users \
  -H "Content-Type: application/json" \
  -d '{"name":"John","email":"john@example.com"}'

# Next GET is a cache miss (must fetch fresh data)
curl http://localhost:5501/api/users
```

Performance Comparison

Scenario	Redis=true	Redis=false
First `GET /users`	Database hit	Database hit
Subsequent `GET /users` (within 5 min)	Redis hit (~1ms)	Database hit (~50-100ms)
`POST user` → `GET /users`	List updated in cache	Database re-queried

Troubleshooting

Redis Service Not Starting

```
# Verify --profile redis flag was included
docker compose --profile dev --profile redis up

# Check service status
docker compose ps
# Should see: app-redis (Running)
```

"Connection Refused" Errors

```
# Verify Redis is healthy
docker logs app-redis
# Should show: Ready to accept connections

# Check Redis connectivity from app
docker logs nodeapi-dev -f
```

Cache Not Updating After Create/Update

- Verify `REDIS_ENABLED=true` in `.env`
- Check logs: `docker logs nodeapi-dev -f` for errors
- Manually clear cache: `docker exec app-redis redis-cli FLUSHDB`

Isolate Database Issues

Set `REDIS_ENABLED=false` to bypass cache entirely and test database directly.

Summary

Aspect	Implementation
Cache Library	`redis` v5.11.0 (Node.js client)
Caching Pattern	Cache-aside (lazy load)
Default TTL	300 seconds (configurable)
Toggleable	Yes (`REDIS_ENABLED` env var)
Docker Integration	Profile-gated (`--profile redis`)
Code Decoupling	Cache logic isolated in `cache.js`

Redis Cache Integration Technical Note | maiproto Project

Document Version 1.0 | Generated March 2026